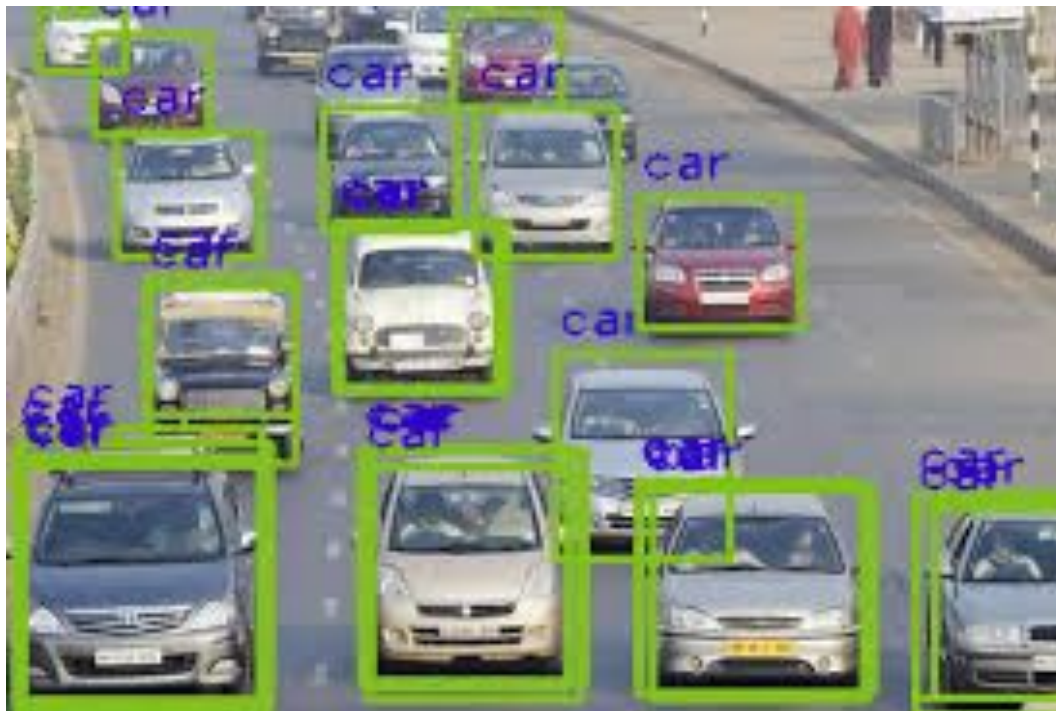


# Детекция и сегментация

Марк Блуменау

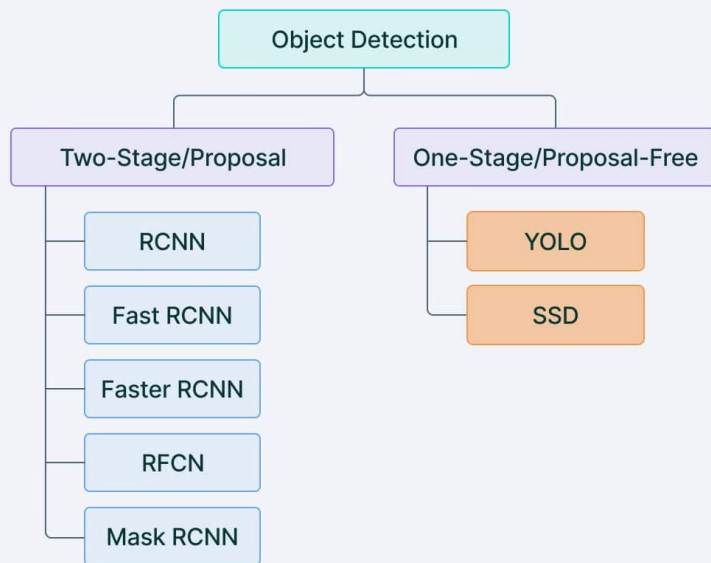
Классификация = бесполезно?

# Что такое детекция?

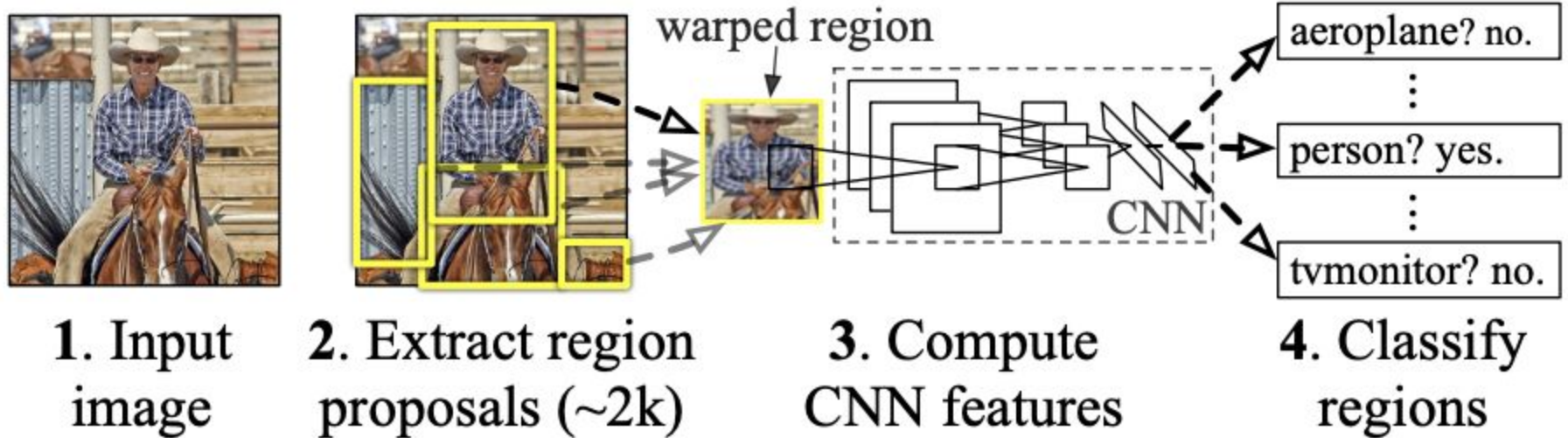


# Какие есть подходы?

## One and two stage detectors

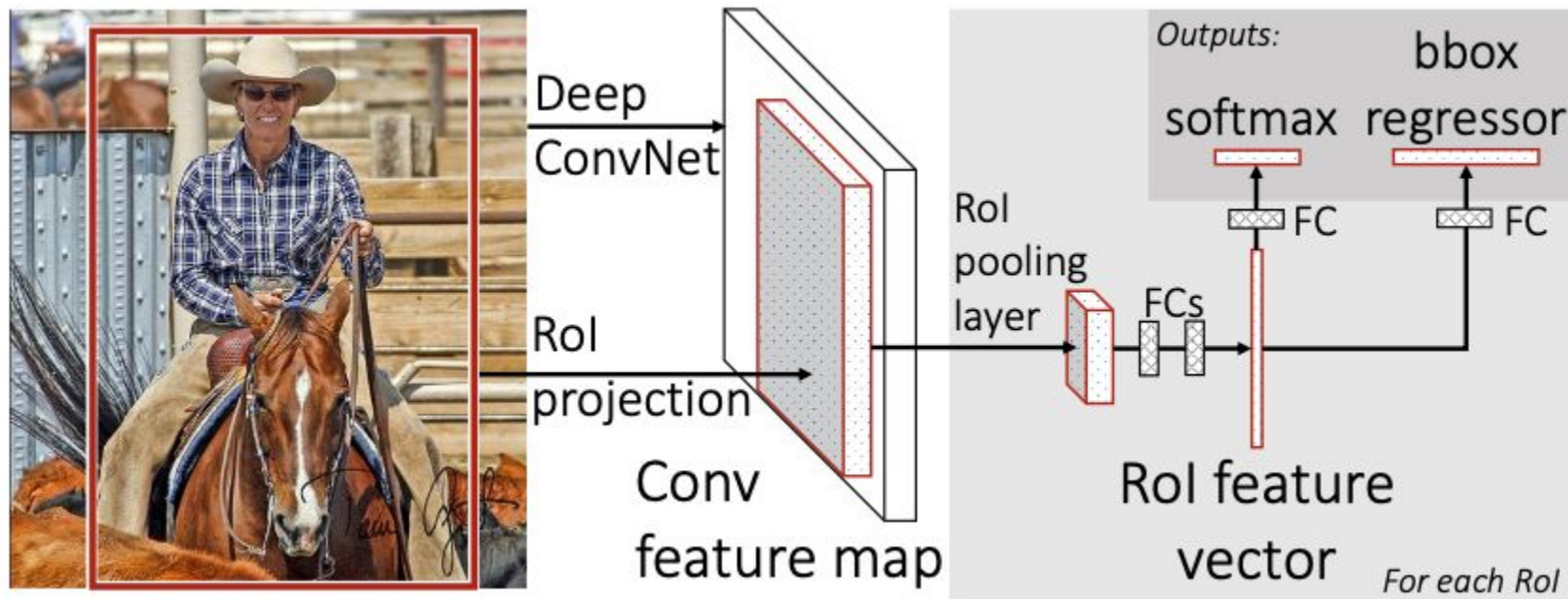


# Two shot (stage) detector: R-CNN 2013

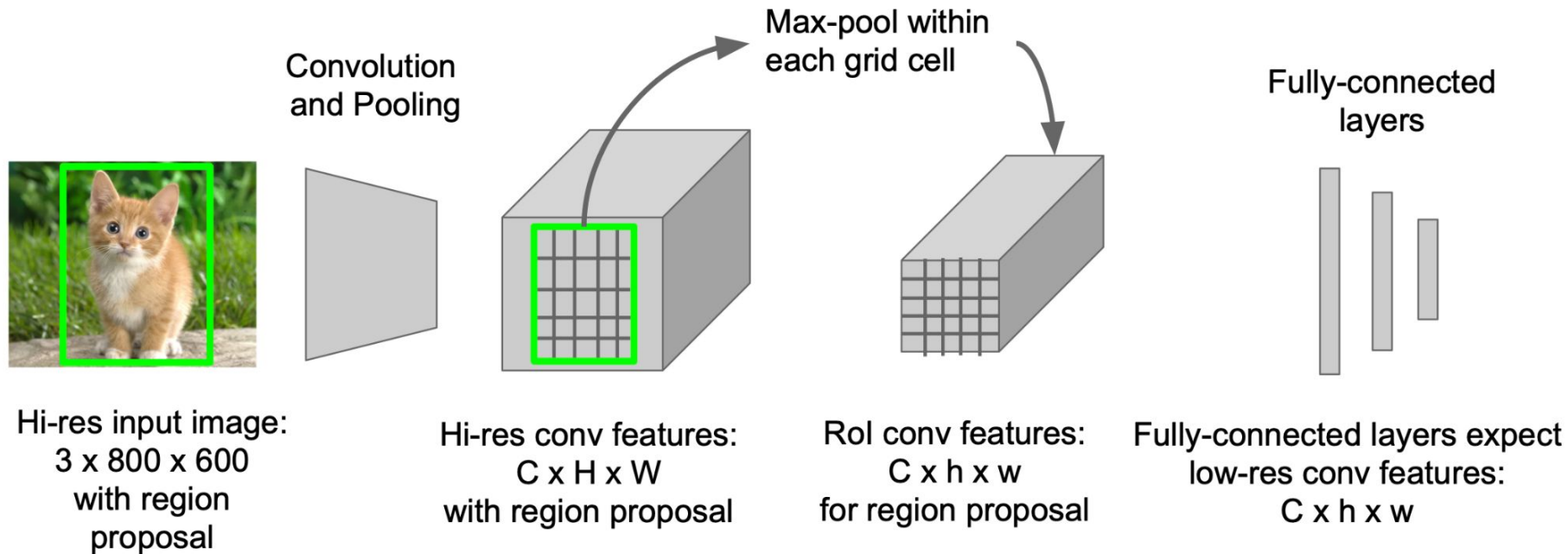


Почему это бред?

# Fast R-CNN 2015: we need to go faster



# ROI Projection





## Наш новый лосс

$$L(p, u, t^u, v) = L_{\text{cls}}(p, u) + \lambda[u \geq 1]L_{\text{loc}}(t^u, v)$$

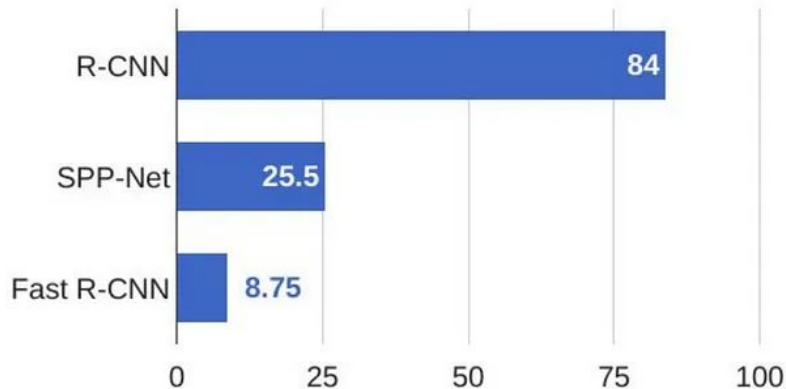
$$L_{\text{loc}}(t^u, v) = \sum_{i \in \{\text{x}, \text{y}, \text{w}, \text{h}\}} \text{smooth}_{L_1}(t_i^u - v_i),$$

in which

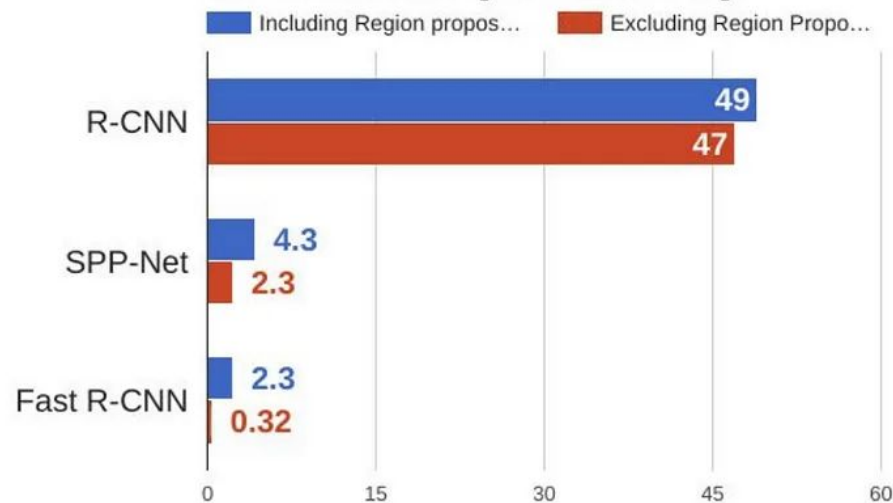
$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise,} \end{cases}$$

# Сравнение скорости работы

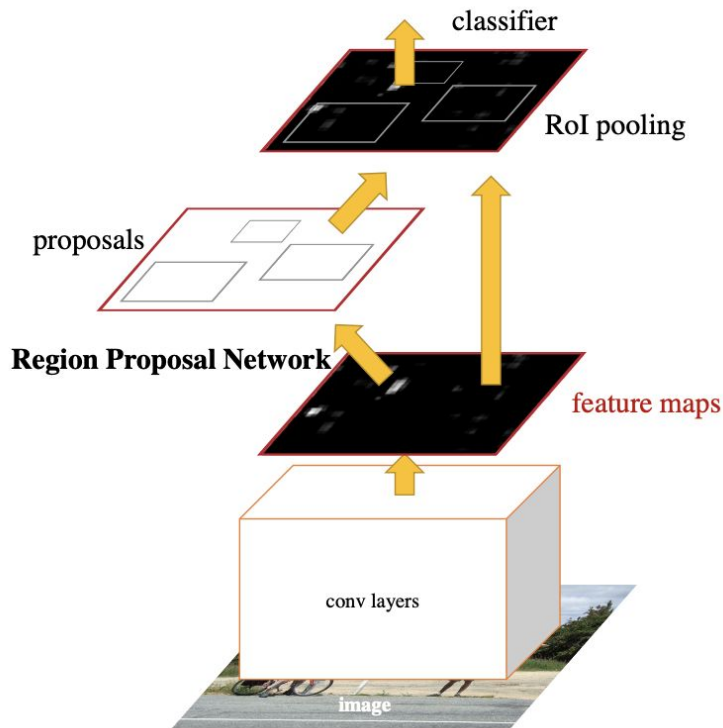
## Training time (Hours)



## Test time (seconds)



# Faster R-CNN 2015 (+2 month): а если выкинуть классику?

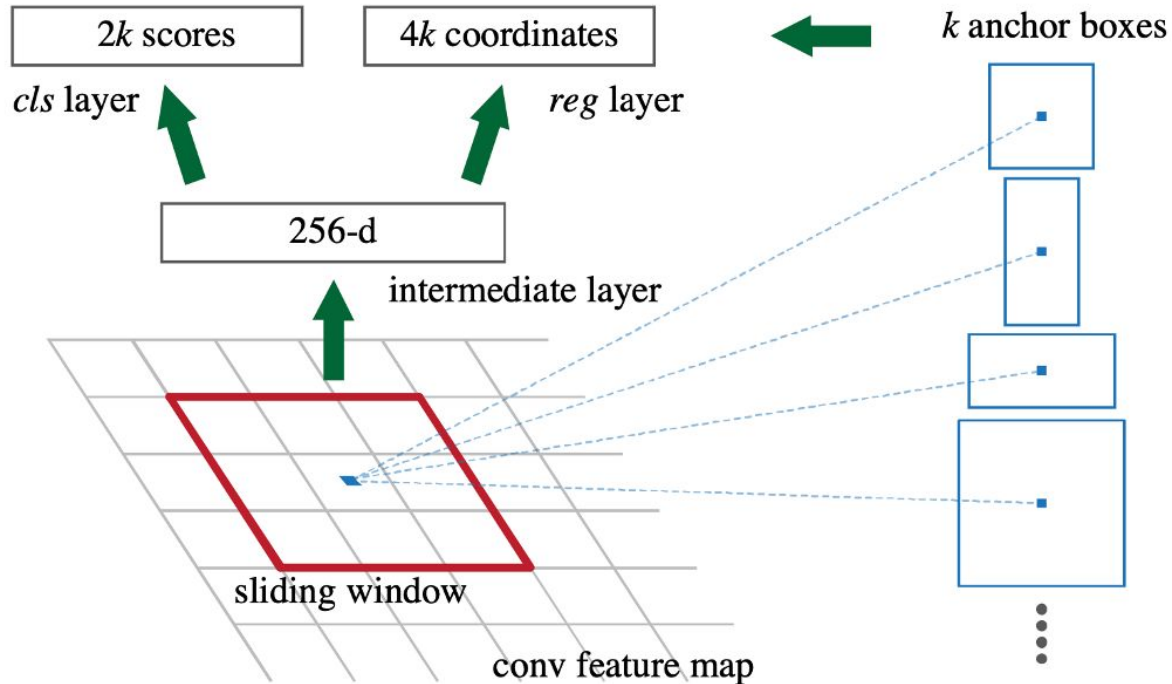


<https://doi.org/10.1109/TPAMI.2016.2577031>

# Region proposal network

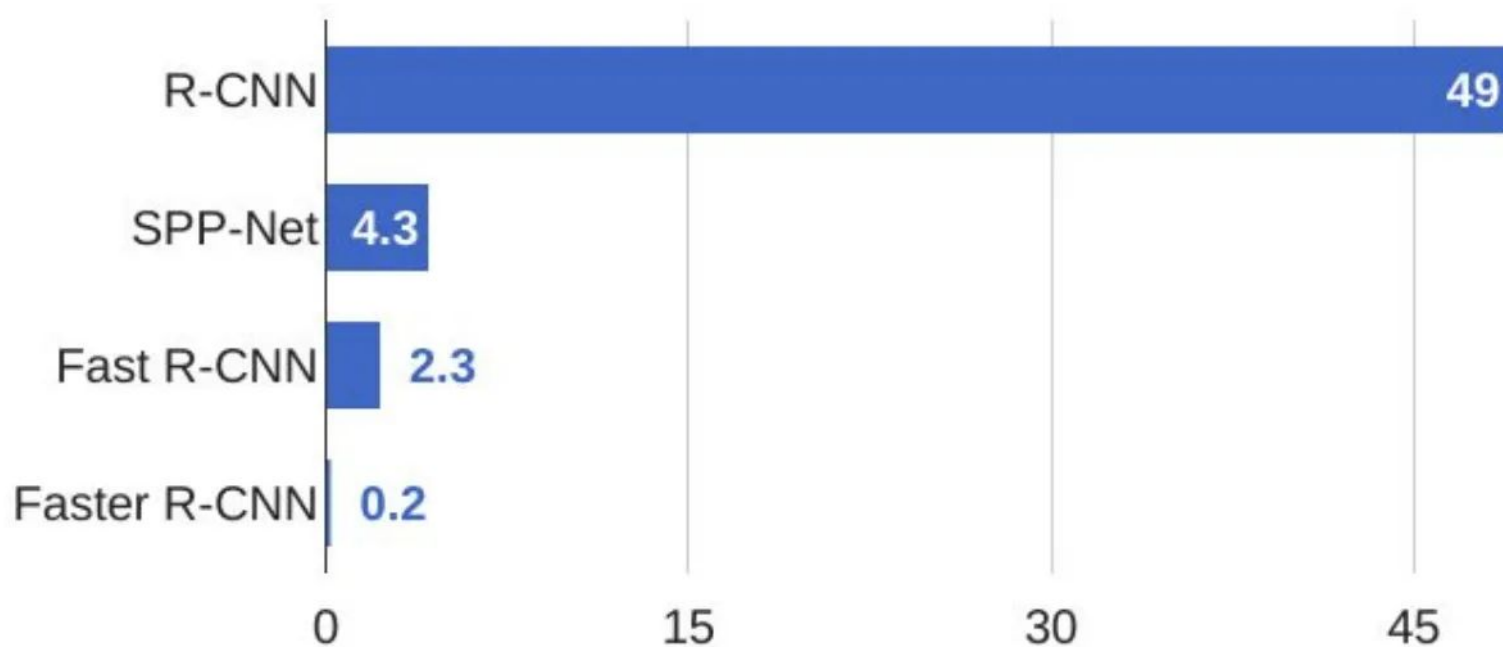
- Берём последний тензор из свёрточной сети
- Применяем свёрточный слой  $3 \times 3$  с большим количеством каналов (256 или 512)
- Для каждой точки берём 9 прямоугольников с разными длинами сторон и соотношениями сторон (якоря)
- Для каждого предсказываем:
  - Есть ли там объект
  - 4 поправки к сторонам

# Region proposal network

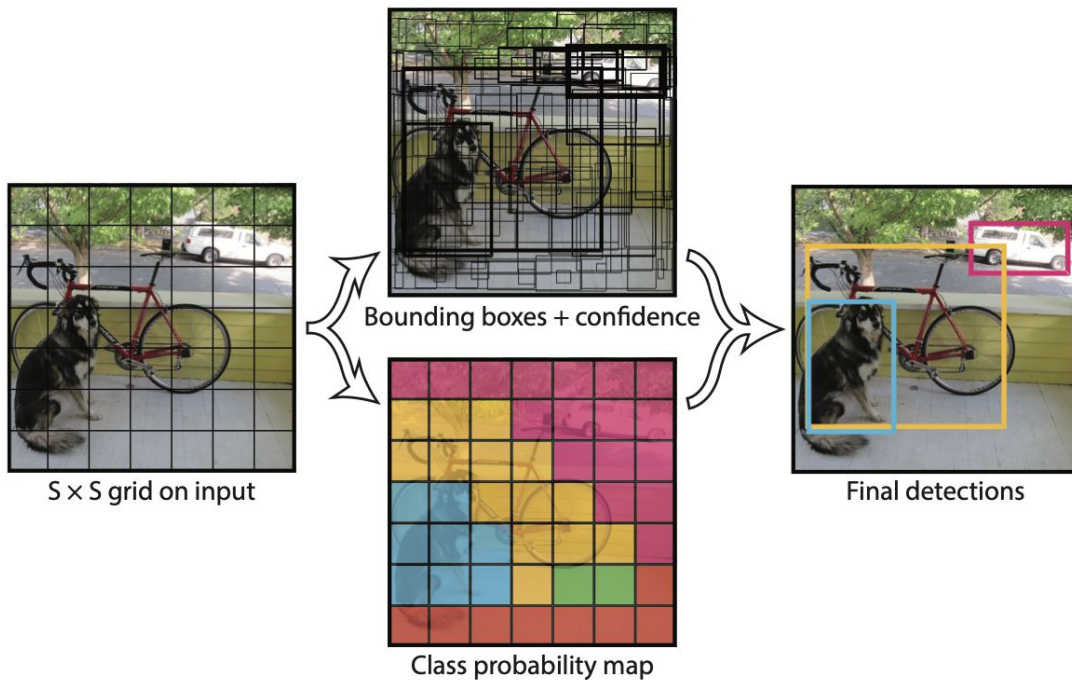


# Выкидываем классику

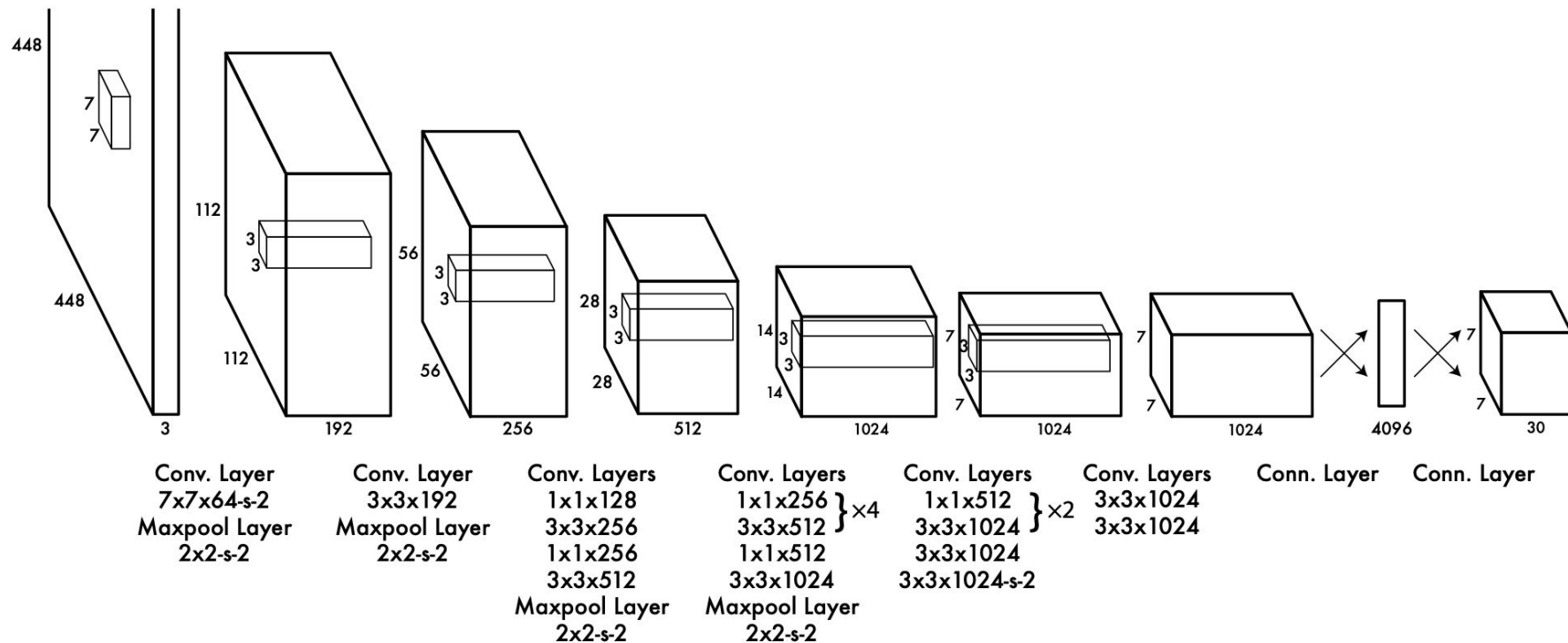
## R-CNN Test-Time Speed



# А ещё быстрее?



# Но как выглядит архитектура?





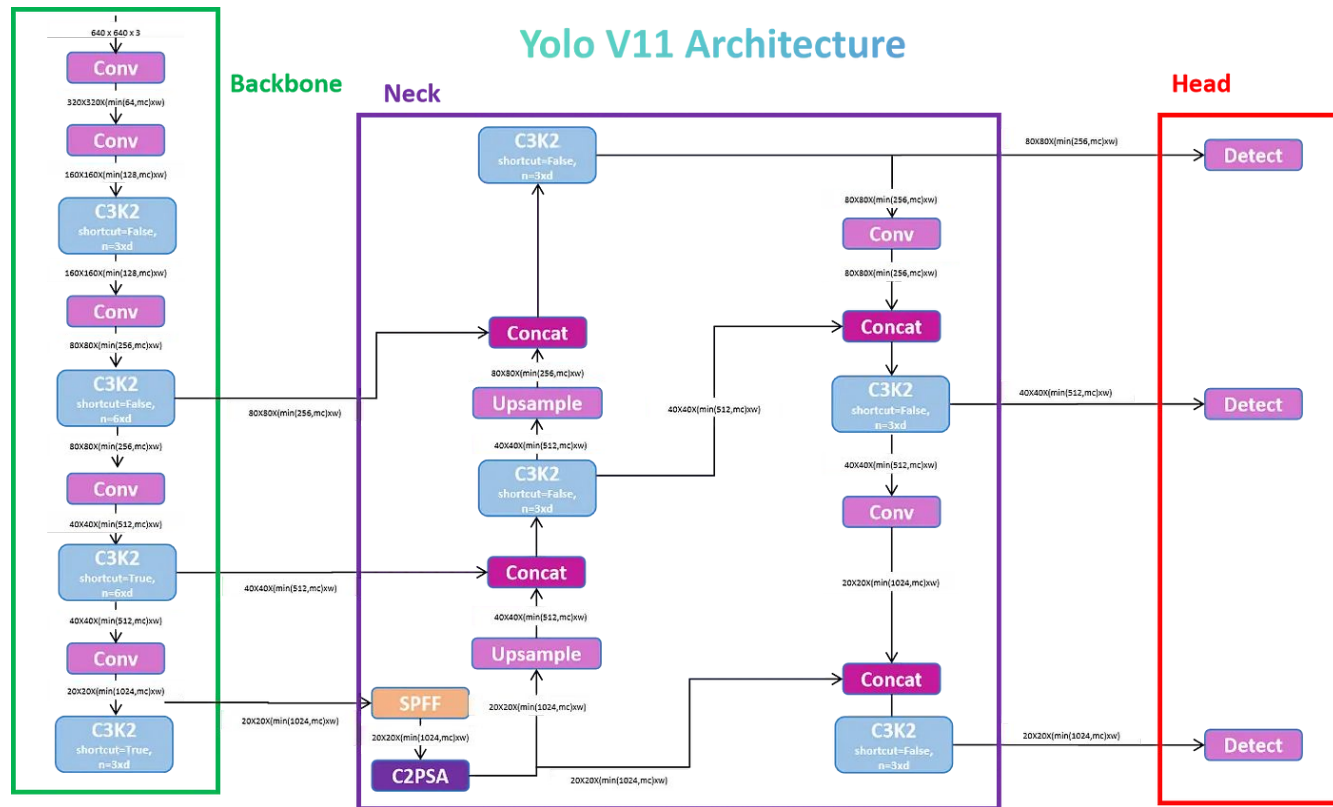
А учить это как?

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[ (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[ \left( \sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left( \sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

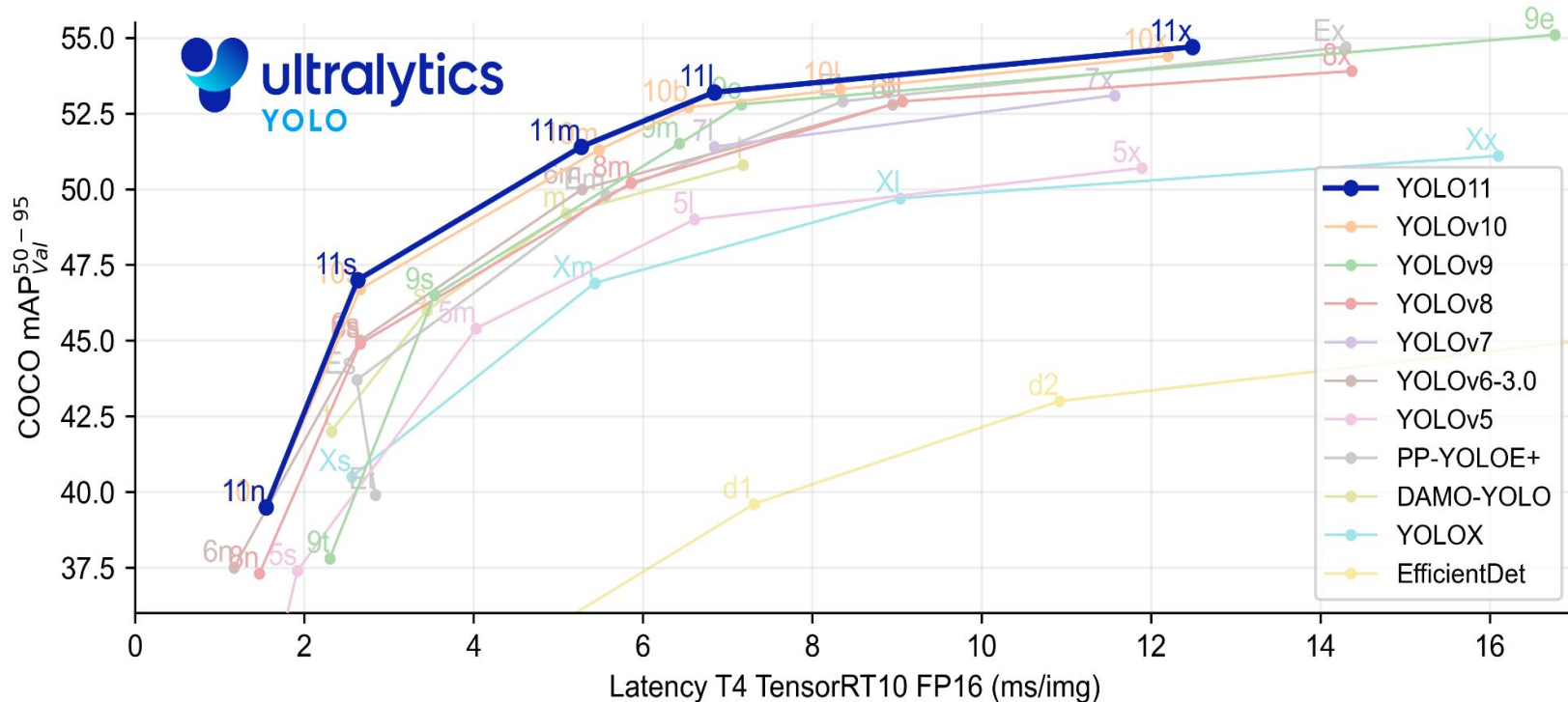
## И зачем мы с этим возились?

| Real-Time Detectors     | Train     | mAP         | FPS        |
|-------------------------|-----------|-------------|------------|
| 100Hz DPM [31]          | 2007      | 16.0        | 100        |
| 30Hz DPM [31]           | 2007      | 26.1        | 30         |
| Fast YOLO               | 2007+2012 | 52.7        | <b>155</b> |
| YOLO                    | 2007+2012 | <b>63.4</b> | 45         |
| Less Than Real-Time     |           |             |            |
| Fastest DPM [38]        | 2007      | 30.4        | 15         |
| R-CNN Minus R [20]      | 2007      | 53.5        | 6          |
| Fast R-CNN [14]         | 2007+2012 | 70.0        | 0.5        |
| Faster R-CNN VGG-16[28] | 2007+2012 | 73.2        | 7          |
| Faster R-CNN ZF [28]    | 2007+2012 | 62.1        | 18         |
| YOLO VGG-16             | 2007+2012 | 66.4        | 21         |

# А что делают сейчас?



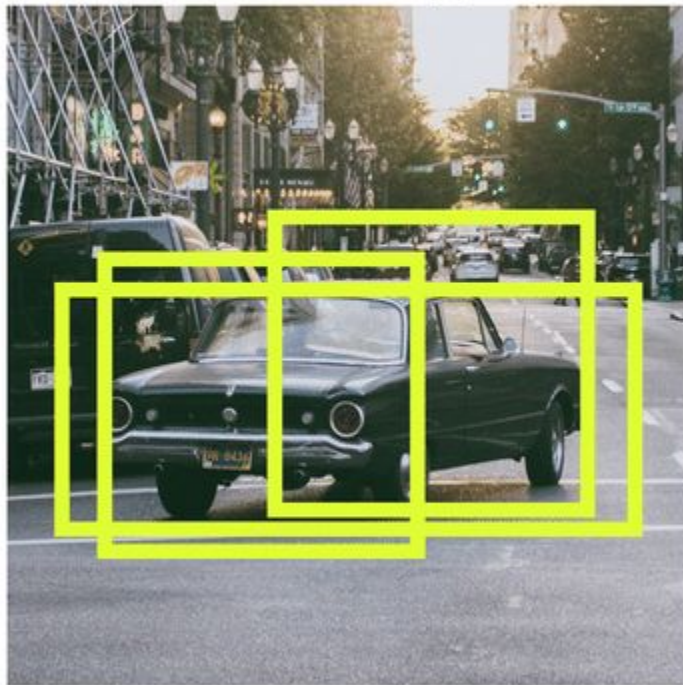
# И что это дает?



<https://github.com/ultralytics/ultralytics>

# Non-maximum suppression (NMS)

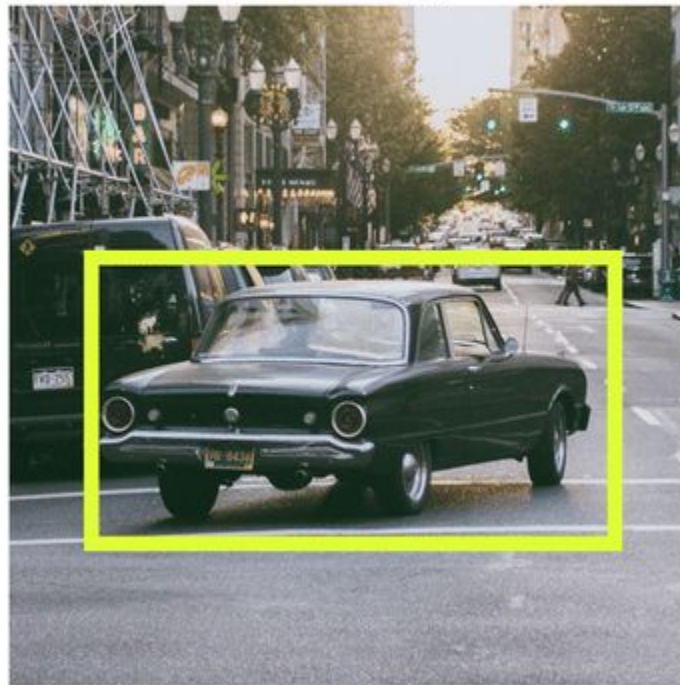
Before non-max suppression



Non-Max  
Suppression



After non-max suppression



## Non-maximum suppression (NMS)

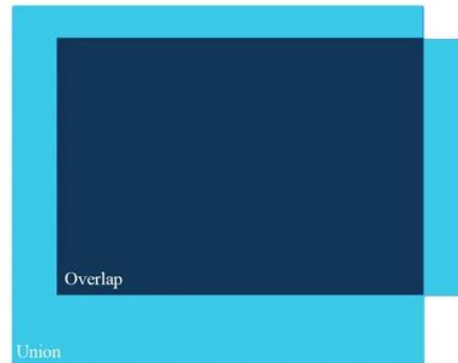
- Модель выдаёт для класса C список прямоугольников с уверенностями
- Проходим в порядке уменьшения уверенности
- Для каждого прямоугольника удаляем все последующие, с которыми Intersection over Union (IoU) > 0.5

# Метрика качества

- 1) Получаем список bbox
- 2) Если  $\text{IoU}(y, \text{pred}) > 0.5 \rightarrow$  bbox корректный
- 3) Если класс детекции корректен тоже, то ставим True Positive (TP)
- 4) Если  $\text{IoU} < 0.5$  или бокс дублируется – False Positive (FP)
- 5) Если  $\text{IoU} > 0.5$ , но класс неверный или bbox вовсе нет – False Negative (FN)



$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}}$$



# PR кривая

Precision =  $TP / (TP + FP)$

Recall =  $TP / (TP + FN)$

AP = Average Precision = AUC PR

mAP = mean Average Precision

